

HKM Edge Plugin

Architecture, end-to-end flow, and complete command reference

Package `alfacode-team/hkm-plugin-edge` · **Namespace** `Plugins\Edge\` · **Solves** `edge.routing` · **Version** 1.0.1 · **Activation** on-demand (CLI-first)
Source `/home/home/Documents/HKMCODE/php/plugins/hkm-plugin-edge`

1. What the plugin is & where it sits
2. Component map (layer by layer)
3. The core flow: probe → decide → render → apply
4. Strategy decision tree
5. The SNI stream splitter
6. Reusing an existing splitter (merge)
7. Sites, domains & the project registry
8. Serve models: PHP-FPM vs OpenSwoole
9. `edge:status` — probe & preview
10. `edge:apply` — render, write, reload
11. `edge:hosts` — `/etc/hosts` sync
12. `edge:service` — systemd / supervisor
13. What a rendered vhost contains
14. Security hardening
15. Configuration reference
16. Safety mechanisms
17. Recipes & troubleshooting
18. Observations from the code read

1 · What the plugin is & where it sits

Edge is a **host control-plane tool**, not a request-serving module. It generates the machine's web-server front configuration (nginx and/or Apache) from the platform's **registered project domains**, adapting to whatever web server is actually running on that host. It then validates the generated config and reloads the server.

Its value is entirely on the CLI. It exposes one published contract so a route could use it, but it declares `"routes": []` and `"requires": []` — nothing loads it on the HTTP path unless a route asks for `edge.routing`.

Zero infrastructure

No ports, no database, no DI. The service is built statically in `Provider::service()`; collaborators read config through the `edge_config()` helper.

CLI-deferred

`boot()` registers the four commands inside `$cli->defer(...)`, so only a CLI process ever constructs them.

Idempotent by design

Every write is atomic (temp + rename), marker-delimited, and merge-based — re-runs never duplicate and never clobber hand-written entries.

GDA placement

Layer	Files	Responsibility
API/Contracts/	EdgeServiceContract	The only published surface: <code>detect</code> , <code>phpFpm</code> , <code>plan</code> , <code>apply</code> , <code>serviceUnits</code> , <code>syncHosts</code> .
Domain/	<code>ServerStack</code> , <code>Strategy</code> , <code>Site</code> , <code>EdgePlan</code> , <code>TlsMode</code> , <code>TlsConfig</code> , <code>ServeModel</code> , <code>CacheProfile</code> , <code>SwooleOptions</code>	Pure value objects. The routing decision lives here (<code>ServerStack::strategy()</code>) — expressed once, not scattered in the CLI.
Application/	EdgeService	Orchestrates: <code>probe</code> → <code>strategy</code> → <code>sites</code> → <code>render</code> → <code>write</code> → <code>validate</code> → <code>reload</code> . Holds no state.
Infrastructure/	<code>SystemProbe</code> , <code>SiteCollector</code> , <code>ConfigRenderrer</code> , <code>HostsFileWriter</code> , <code>StreamConfigWriter</code> , <code>ServiceRenderrer</code>	All OS contact and all text generation. <code>ConfigRenderrer</code> and <code>ServiceRenderrer</code> are pure (no I/O).
Infrastructure/Cli/	4 <code>AbstractCommand</code> subclasses	Flag parsing + human reporting only. All logic delegates to the contract.

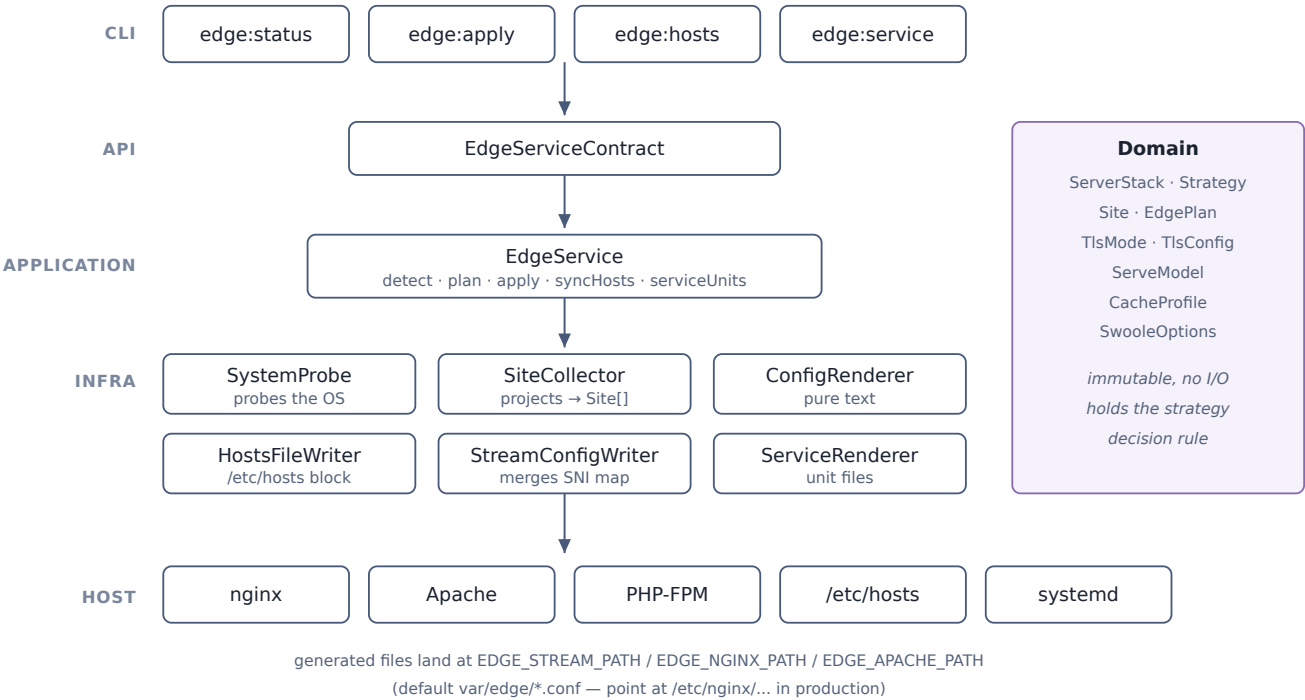


Figure 1 — Component map. CLI is a thin shell; the strategy rule lives in the Domain.

2 · The core flow: probe → decide → render → apply

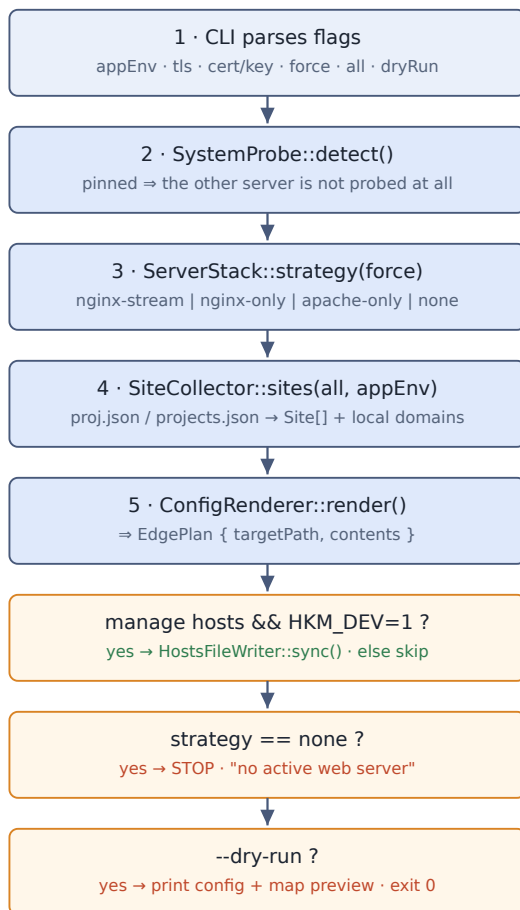
Everything in the plugin funnels through two methods on `EdgeService` . `plan()` is read-only and produces an `EdgePlan` ; `apply()` calls `plan()` and then performs the writes. `edge:status` stops after `plan()` .

```

plan(all, tlsMode, sslCert, sslKey, appEnv, force)
├─ forcedStrategy(force)                → ?Strategy    (operator override, resolved FIRST)
├─ SystemProbe::detect(nginxOnly, apacheOnly) → ServerStack (a pinned run never probes the other server)
├─ ServerStack::strategy(forced)         → Strategy      (the decision, in the Domain)
├─ reuseStream = NginxStream && EDGE_REUSE_STREAM && stack.nginxHasStreamConfig
├─ CacheProfile::fromAppEnv(appEnv)      → dev | production
├─ SiteCollector::sites(all, appEnv)     → Site[]        (docroot, domains, upstream, env)
└─ ConfigRenderer::render(...)          → [targetPath, contents]
                                         ⇒ EdgePlan

```

PLAN (read-only — this is all edge:status runs)



APPLY (writes)



no

Figure 2 — `edge:apply` end to end. Amber blocks are gates; red text marks a terminal outcome.

3 · Strategy decision tree

The rule lives in `ServerStack::strategy(?Strategy $force)` — one method, one place. A non-null `$force` (from `--nginx-only` / `--apache-only` or `EDGE_FORCE_STRATEGY`) short-circuits detection entirely and pins a single server **with no fallback**.

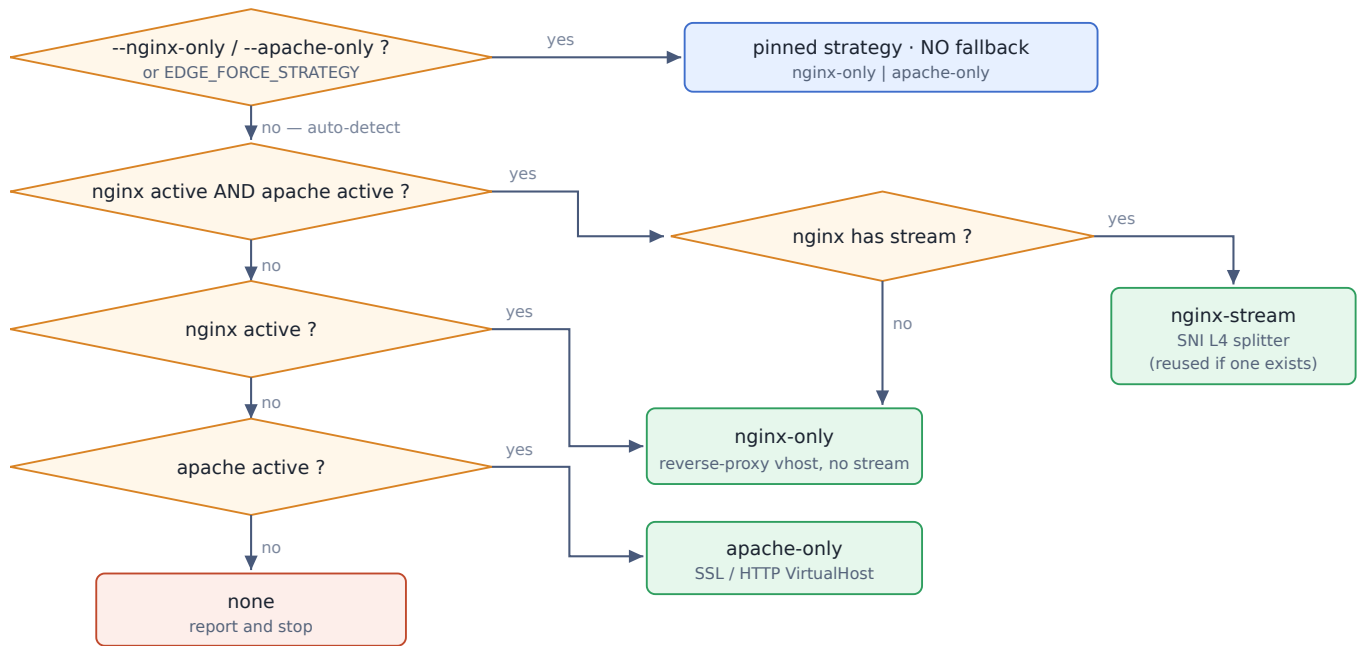
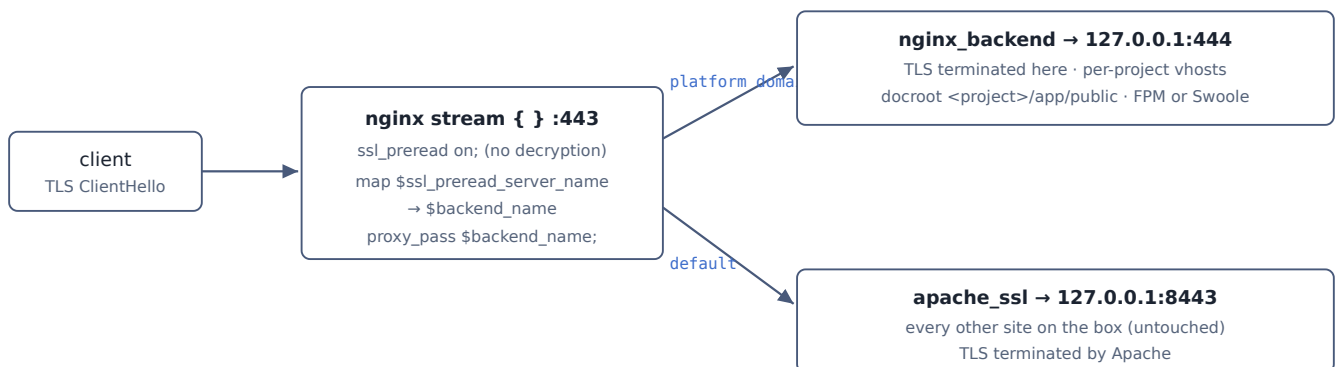


Figure 3 — `ServerStack::strategy()` . nginx is always the front when it is running.

Detected stack	Strategy	What gets rendered · where
nginx + Apache active, nginx has <code>stream</code>	nginx-stream	SNI (L4) splitter + internal backend vhosts (TLS on :444) → <code>EDGE_STREAM_PATH</code>
... and nginx already has a splitter	nginx-stream (reuse)	Only the internal vhosts; the platform's domains are merged into the <i>existing</i> map in place
only nginx active, or Apache inactive, or nginx lacks <code>stream</code>	nginx-only	Plain nginx vhost, no stream layer → <code>EDGE_NGINX_PATH</code>
only Apache active	apache-only	Apache SSL/HTTP <code>VirtualHost</code> → <code>EDGE_APACHE_PATH</code>
neither active	none	Nothing. Only the hosts sync (dev) runs.

4 · The SNI stream splitter

When both servers run, nginx owns `:443` at layer 4. `ssl_preread` reads the TLS ClientHello's SNI **without decrypting**, the `map` picks a backend, and the raw TLS stream is forwarded. TLS is terminated by the backend — the stream layer never sees plaintext, so certificates stay on the backends.



`EDGE_NGINX_BACKEND` / `EDGE_APACHE_BACKEND` set the two upstreams · `EDGE_LISTEN_PORT` the public port

Figure 4 — Layer-4 SNI routing. Certificates live on the backends, never on the splitter.

```

stream {
    upstream nginx_backend { server 127.0.0.1:444; }
    upstream apache_ssl    { server 127.0.0.1:8443; }

    map $ssl_preread_server_name $backend_name {
        app.example.com    nginx_backend;
        default apache_ssl;
    }

    server { listen 443; proxy_pass $backend_name; ssl_preread on; }
}

```

Placement. The `stream {}` block must be included at the nginx **main** context (top level of `nginx.conf`), **not** inside `http {}`.

4.1 · Reusing an existing splitter (the merge path)

If `nginx -T` shows the running nginx already declares an SNI splitter — a `map $ssl_preread_server_name ... plus ssl_preread`, in a file Edge does not manage — Edge will **not** write a second, conflicting one. Instead it emits only the internal backend vhosts and merges the platform's public domains into *your* map, in place.

Step (<code>StreamConfigWriter::merge</code>)	Behaviour
Locate the file	<code>SystemProbe::nginxStreamConfigFile()</code> parses <code>nginx -T</code> per <code># configuration file <path>:</code> marker and skips Edge's own managed file, so re-runs never match themselves.
Isolate the map	Regex finds the <code>map</code> opener, then brace-matching finds its true close. Anything outside the map is never modified.
Strip the old managed sub-block	The previous <code># >>> HKM Edge (managed domains) >>> ...</code> block is removed, so the sub-block is rewritten whole each run — no duplicates.
Diff	Domains already keyed anywhere in the map (yours or ours) go to <code>present</code> and are left alone; the rest become <code>added</code> .
Insert	The managed sub-block is placed immediately before the default line (or at the end when there is none), indent- and column-aligned to your existing entries.
Write	Atomic temp + rename. On failure the whole <code>edge:apply</code> fails loudly — a missing <code>sudo</code> must never report success.

```

map $ssl_preread_server_name $backend_name {
    migratetravel.com          nginx_backend;    # yours — untouched
    # >>> HKM Edge (managed domains) >>>
    app.showmeuganda.com      nginx_backend;    # added by edge:apply
    admin.hkmvote.com         nginx_backend;
    # <<< HKM Edge (managed domains) <<<
    default                   apache_ssl;
}

```

Controls: `EDGE_REUSE_STREAM=0` disables reuse (Edge writes its own block instead); `EDGE_STREAM_BACKEND` changes the upstream name the merged domains point at. Preview the exact diff with `edge:apply --dry-run`.

Port side-effect. Because an existing splitter owns `:443`, the `nginx-only` vhost auto-detects this (`ServerStack::nginxHasStreamConfig`) and listens on the *internal* backend port instead — otherwise nginx would fail with “Address already in use”. Force it with `EDGE_BEHIND_SNI_ROUTER=1` or pin it with `EDGE_NGINX_SSL_PORT`.

5 · Sites, domains & the project registry

`SiteCollector` turns projects into `Site` value objects. Scope is **the current project by default**; `--all` walks the global registry.

Scope	Source	Notes
default (current)	<code>base_path()/proj.json</code>	Name falls back to the directory basename.
<code>--all</code>	<code>projects.json</code> at <code>edge_config('projects_registry')</code>	Resolved from <code>EDGE_PROJECTS_DIR</code> → <code>PSP_PROJECTS_DIR</code> → <code>HKM_KERNEL_HOME/projects</code> → <code>base_path('projects')</code> . Edge is a host tool, so it reads the <i>global</i> registry, not the per-project path.

Domain classification

Domains come from `proj.json domains[]`, plus `EDGE_EXTRA_DOMAINS`, minus `EDGE_EXCLUDE_DOMAINS`. Each hostname is **strictly validated** against a charset regex before it can reach a rendered config, so a malformed entry can never inject directives. Then it is split:

Class	Test	Destination
public	Real FQDN (<code>app.example.com</code>)	The server config (stream map / vhost / VirtualHost)
local	TLD in <code>EDGE_LOCAL_TLDS</code> (<code>local</code> , <code>test</code> , <code>localhost</code> , <code>example</code> , <code>invalid</code>) or a single-label host such as <code>myapp</code>	<code>/etc/hosts</code> → loopback. Kept out of the public server config when the site <i>also</i> has public domains...

...with two exceptions. (1) Local-only projects always serve. If a project's domains are *all* local there is nothing else it could serve, so they are rendered into the vhost in every mode — no flag, no env var. Without this a project like `hkmvote` (four `.local` domains) rendered a config with no `server {}` block at all while still reporting “1 site(s)”, which reads as a broken tool rather than a dev-domain project. **(2) Dev mode mixes them in.** When the launcher exports `HKM_DEV=1` (`hkm ... --dev`) — or `EDGE_LOCAL_IN_SERVER=true` — local domains are *also* added to a site that already has public ones. Local domains still go to `/etc/hosts` in every case.

The `Site` value object

Field	Meaning
<code>name</code>	Project name (also the unit name and per-site log filename).
<code>root</code> / <code>docroot</code>	Project root; docroot is always <code><root>/app/public</code> — never the project root, keeping <code>.env</code> , <code>src</code> , <code>vendor</code> out of the web tree.
<code>publicDomains</code> / <code>localDomains</code>	The two classified sets.
<code>model</code> / <code>upstream</code>	<code>fpm</code> → fastcgi socket · <code>swoole</code> → <code>host:port</code> .
<code>env</code>	Run-env injected into the vhost (see below).
<code>swoole</code>	<code>SwooleOptions</code> (ports, ws/health paths, workers, balancing, keepalive) — null for FPM.

Why the vhost carries `env`

FPM workers do not inherit your shell or the `hkm` launcher's environment, so the vhost must carry whatever the launcher exported, as `fastcgi_param` (nginx) / `SetEnv` (Apache):

- `APP_ENV` (from the env flag, else `EDGE_APP_ENV` / `APP_ENV`, else `production`).
- Pass-through of `kernel_env_keys` — `HKM_KERNEL_HOME`, `HKM_DEV_HOME`, `HKM_USERDATA_DIR`, `PSP_GLOBAL_AUTOLOAD`, `PSP_PROJECTS_DIR` — but **only those actually set**. Edge passes through; it never derives them. Disable with `EDGE_INJECT_KERNEL_ENV=false`.

3. Per-project extras from `proj.json` `edge.env` — these win.

`proj.json` — the per-project override block

```
{
  "name": "shop",
  "domains": ["shop.example.com", "shop.local"],
  "edge": {
    "runtime": "openswoole",           // or "php-fpm" (legacy key: "serve")
    "socket": "unix:/run/php/php8.4-fpm.sock", // fpm model only
    "openswoole": {
      "host": "127.0.0.1",
      "ports": [9601, 9602],           // first = primary, rest = extra upstream backends
      "websocket": "/ws",              // null disables the dedicated block
      "health": "/health",
      "php": "/usr/bin/php",
      "command": "app/swoole/index.php",
      "workers": "auto",
      "balance": "least_conn", "max_fails": 3, "fail_timeout": "30s",
      "keepalive": 32, "keepalive_timeout": "60s", "keepalive_requests": 1000
    },
    "env": { "APP_ENV": "production", "SHOP_FLAG": "1" }
  }
}
```

Accepted runtime spellings (case/spacing-insensitive): `php-fpm`, `php_fpm`, `fpm`, `openswoole`, `open-swoole`, `swoole`.

6 • Serve models: PHP-FPM vs OpenSwoole

A • ServeModel::Fpm



B • ServeModel::Swoole

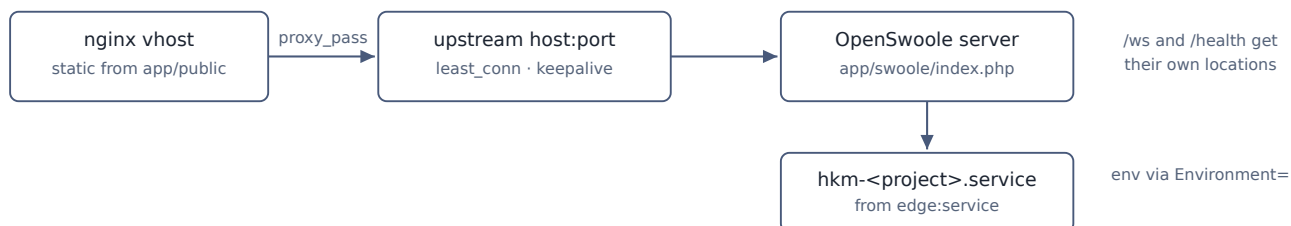


Figure 5 — Both models serve static assets straight off disk; only the dynamic path differs.

	PHP-FPM	OpenSwoole
Upstream	Auto-resolved socket for the running CLI PHP version, or <code>EDGE_FPM_SOCKET</code> , or <code>proj.json edge.socket</code>	<code>host:port</code> from <code>SwooleOptions</code> ; extra <code>ports[]</code> become additional upstream servers
Run env	<code>fastcgi_param</code> lines in the vhost	<code>Environment=</code> lines in the service unit (plus <code>SWOOLE_HOST</code> / <code>SWOOLE_PORT</code> / <code>SWOOLE_WORKERS</code>)
Process supervision	php-fpm already does it — <code>edge:service</code> yields nothing	<code>edge:service</code> renders a systemd unit or supervisor program
Extra nginx blocks	—	<code>map ... \$connection_upgrade</code> (emitted once per file), optional <code>/ws</code> and health locations

FPM socket resolution order: versioned socket for the CLI PHP version → any versioned socket (exact version, else newest) → generic socket (RHEL/custom) → `127.0.0.1:9000`. This is what makes a multi-PHP host bind the vhost to the right pool.

7 · Command reference

Invocation form for every command:

```
hkm cli -p <project> <edge:command> [flags]
```

- Every command scopes to the **current project** by default; `--all` covers every registered project.
- Writing `/etc/nginx`, `/etc/hosts` or `/etc/systemd` needs **sudo**.
- `--dev` is consumed by the `hkm` launcher (it selects the dev kernel and exports `HKM_DEV=1`) and stripped before the command runs. Inside `edge:apply` / `edge:service` use `--local` for `APP_ENV`.

7.1 · `edge:status` READ-ONLY

Probes the host and prints the plan that *would* be applied. Touches nothing — it calls `plan()` and stops.

Flag	Effect
<code>--all</code>	Every registered project instead of only the current one
<code>--nginx-only</code>	Preview the pinned nginx-only strategy
<code>--apache-only</code>	Preview the pinned apache-only strategy

Flow

```
flags → plan(all, force)
  → detect()           nginx installed/active/stream/stream-cfg/brotli, apache installed/active
  → strategy(force)
  → phpFpm()           CLI version · resolved socket · active php*-fpm units
  → sites()             per project: model → upstream → public domains
  → render()            (in memory only)
prints: stack · strategy label (+ "reusing the existing splitter") · sites · local domains · target path
a site with no usable domain is flagged "(NO domains to serve – this site renders no vhost)",
and if NOTHING would render you get an explicit warning instead of a silent empty plan.
--nginx-only / --apache-only also suppress probing the other server, so the reported
stack matches the plan (no "apache installed: yes" on a run that will never touch it).
```

Exit codes: `SUCCESS`; `INVALID` if both `--nginx-only` and `--apache-only` are passed.

7.2 · `edge:apply` WRITES

The main command: detect → render → write atomically → validate → reload. Full sequence in **Figure 2**.

Reporting

The header line reads `strategy: nginx-only → /path (2 site(s))`, or `(3 site(s), 1 served)` when a project in scope produced no vhost. If **nothing** was rendered the command says so and lists the causes — an empty config file is never printed without an explanation.

Scope & write flags

Flag	Effect
<code>--dry-run</code>	Print exactly what would be written (config + stream map preview); change nothing
<code>--no-reload</code>	Write the file but skip <code>nginx -t / reload</code>
<code>--no-hosts</code>	Skip the <code>/etc/hosts</code> step
<code>--all</code>	Render every registered project into one file

Environment → cache profile (pick at most one)

Flag	APP_ENV	Cache profile
<code>--local</code> (or <code>--dev</code>)	<code>local</code>	DEVELOPMENT — everything <code>no-store</code> ; also defaults TLS to none
<code>--development</code> / <code>-d</code>	<code>development</code>	DEVELOPMENT
<code>--production</code>	<code>production</code>	PRODUCTION — immutable long-lived static assets, HTML never cached
<i>none given</i>	configured <code>APP_ENV</code> / <code>EDGE_APP_ENV</code>	Anything unrecognised → DEVELOPMENT, never production (fail-safe)

TLS mode

Flag	Emits
<code>--tls=ssl</code>	One <code>listen 443 ssl</code> server (HTTP/2, TLS pinning, HSTS). No <code>:80</code> . Needs a certificate.
<code>--tls=none</code> / <code>--no-ssl</code>	One <code>listen 80</code> server, plain HTTP, no certificate, no HSTS.
<code>--tls=both</code>	Two servers: <code>:80</code> serves the ACME challenge then <code>301 s</code> to HTTPS, plus the <code>:443</code> block.
<code>--ssl-cert=</code> / <code>--ssl-key=</code>	Override <code>ssl.cert</code> / <code>ssl.key</code> for this run.

The **both** redirect always lets `/.well-known/acme-challenge/` through *before* redirecting, so certificates can still be issued and renewed over plain `:80` . Under the `nginx-stream` strategy the internal vhosts always terminate TLS on the internal port — the TLS mode applies to the single-server strategies.

Strategy override

Flag	Effect
<code>--nginx-only</code>	nginx serves everything, no Apache fallback — plain reverse-proxy vhost, no stream layer
<code>--apache-only</code>	Apache serves everything, no fallback
<i>neither</i>	Auto-detect (Figure 3). Deploy default: <code>EDGE_FORCE_STRATEGY</code> .

Failure & rollback behaviour

Situation	Result
Target directory missing and cannot be created	<code>ok=false</code> , nothing written
Existing config cannot be backed up	Refuses to overwrite — <code>ok=false</code>
Write/rename fails	temp file and backup removed, <code>ok=false</code>
Stream map merge fails (usually no sudo)	Whole apply fails — never reports success
<code>nginx -t</code> / <code>configtest</code> fails	Rolls back to the backup, reports the test output, exit 1. If the restore itself fails you get an explicit WARNING to restore manually.
Reload command fails	<code>ok=false</code> with the reload output (config already validated, so it stays)
Success	Backup deleted — no stale <code>.bak-*</code> beside the live config

Why the backup matters. The new file is written over the live path and validated *after*. Without the backup, a failed `nginx -t` would leave a broken config in place and the next reload by anyone — a deploy, logrotate, a reboot — would load it, surfacing the failure later and elsewhere.

7.3 • `edge:hosts` WRITES /etc/hosts DEV ONLY

Writes the local (`.local` / `.test` /...) domains to the loopback inside a marker-delimited block. The rest of the hosts file is never touched.

Flag	Effect
<code>--dry-run</code>	Show what would change; write nothing (prints the block)
<code>--remove</code>	Remove the HKM-managed block entirely (never your own entries)
<code>--all</code>	Every registered project
<code>--force</code>	Allow running outside dev mode

The two rules

1. **Dev gate.** `/etc/hosts` is a developer-machine concern — a live server resolves public domains through DNS. Without `HKM_DEV=1` (exported by `hkm ... --dev`) the command **refuses** with an explanation, and `edge:apply` **silently skips** the hosts step. `--force` overrides for `edge:hosts` only.
2. **Existing entries win.** Every domain is checked against the rest of the file (comments and multi-host lines handled). A hostname mapped by hand elsewhere is reported as *skipped* and left authoritative; only genuinely missing ones are added.

Merge semantics

```
read /etc/hosts
├─ parseBlock()    domains already inside our markers    → seed
├─ existingHosts() every host mapped OUTSIDE the block    → skip list
├─ merge           seed + new (order preserved; a no-op run is byte-identical)
└─ atomic write    <path>.hkm.tmp → rename

# >>> HKM Edge (local domains) >>>
127.0.0.1    api.hkm.local
127.0.0.1    hkm.local
# <<< HKM Edge (local domains) <<<
```

Because the merge seeds from the block's current contents, syncing one project never drops another project's entries from an earlier run.

7.4 • `edge:service` WRITES ONLY WITH `--write`

Renders the process-manager unit that keeps a project's OpenSwoole server alive behind the reverse proxy. **PHP-FPM projects yield nothing** — php-fpm supervises those workers.

Flag	Effect
<code>--supervisor</code>	Render a supervisor <code>[program:...]</code> block instead of a systemd unit
<code>--all</code>	Every registered project
<code>--user=<name></code>	User/group the service runs as (default <code>www-data</code>)
<code>--write[=dir]</code>	Write to disk; default <code>/etc/systemd/system</code> or <code>/etc/supervisor/conf.d</code>
<code>--local / -d / --production ...</code>	Same APP_ENV flags as <code>edge:apply</code> (they feed the unit's <code>Environment=</code>)

Flow

```
serviceUnits(format, all, appEnv, user)
└─ for each Site
    └─ ServiceRenderer::supports() → skip unless ServeModel::Swoole
    └─ unitName() → hkm-<project> (non [a-z0-9._-] → '-')
    └─ systemd() | supervisor()
        ExecStart = <php> <root>/<command> (absolute – never depends on $PATH)
        Environment = site env + SWOOLE_HOST + SWOOLE_PORT [+ SWOOLE_WORKERS]
        Restart=always · RestartSec=5 · KillSignal=SIGTERM · TimeoutStopSec=30

no --write → print to stdout
with --write → <dir>/hkm-<project>.service (or .conf), then:
    systemctl daemon-reload && systemctl enable --now <unit>
    supervisorctl reread && supervisorctl update
```

With no OpenSwoole project in scope the command warns and exits 0, pointing at `proj.json edge.runtime = "openswoole"`.

8 • What a rendered vhost contains

`ConfigRenderer` is pure text generation. `vhostCommon()` builds every shared fragment in one place so the FPM and Swoole vhosts can never drift apart.

Fragment	Behaviour
File-level prelude	Emitted once per file when <code>EDGE_HTTP_PRELUDE=1</code> : <code>log_format</code> , rate-limit zones, Cloudflare real-IP ranges. Opt-in, because re-declaring what your <code>nginx.conf</code> already has is a duplicate-definition error.
CORS map / upgrade map	Also once per file — nginx rejects a duplicate <code>map</code> for the same variable. The <code>\$connection_upgrade</code> map only appears when a Swoole site is present.
Per-site logs	<code><name>.access.log</code> / <code>.error.log</code> in both profiles (<code>EDGE_PER_SITE_LOGS=0</code> falls back to the global log). <code>error_log ... debug</code> is opt-in: it needs an nginx built <code>--with-debug</code> , is enormous, and can log tokens.
Header set	CORS + security headers + HSTS, built at both indents and re-emitted in every location that declares any <code>add_header</code> — one <code>add_header</code> in a location drops all inherited ones.
CORS policy	<code>off</code> (default) · <code>allowlist</code> (origins echoed via a <code>\$http_origin</code> map) · <code>wildcard</code> (opt-in). An allowlist with no origins degrades to <code>off</code> .
Static assets	DEV: <code>no-store</code> + <code>access_log off</code> , and <code>map / json</code> are served. PROD: <code>expires 1y</code> , <code>public</code> , <code>immutable</code> , and <code>map / json</code> are dropped from the served set (source maps expose original source). A miss is a hard <code>404</code> , never forwarded to the app.
Dynamic responses	Never cached unless <code>EDGE_CACHE_HTML=1</code> . Dev adds <code>Pragma / Expires</code> too.
Deny list	<code>EDGE_DENY_DIRS</code> (default <code>vendor,node_modules,tests,.git,.github,bootstrap/cache</code>), prefix-matched before the static rule. <code>storage</code> is deliberately excluded (the <code>public/storage</code> upload symlink).
Method guard	<code>EDGE_ALLOWED_METHODS</code> → <code>405</code> otherwise; empty disables.
Rate limiting	<code>limit_req</code> / <code>limit_conn</code> are emitted only inside the location that runs the application — <code>location = /index.php</code> for PHP-FPM, <code>location /</code> (the proxy) for OpenSwoole — never at server scope. At server scope one page load pulling 30 assets burns 31 tokens against the zone, so a real visitor gets 429s on CSS/JS while the expensive dynamic request goes unprotected. Static files are never throttled. Requires the prelude (<code>EDGE_HTTP_PRELUDE=1</code>), which declares the zones.
Compression	<code>auto</code> resolves per server's real capability: brotli when the module is present, else gzip. An explicit <code>brotli</code> also degrades to gzip, so the emitted config never fails the server's own config test.
HSTS	TLS modes only. PRODUCTION uses <code>EDGE_HSTS_MAX_AGE</code> (1y); DEVELOPMENT is forced short (<code>EDGE_HSTS_DEV_MAX_AGE</code> , no subdomains, no preload).
Dev-only locations	<code>robots.txt</code> (Disallow all) and <code>/nginx-status</code> . <code>/nginx-status</code> is gated on the true development profile, never a mere <code>EDGE_DEV_VHOST</code> override — behind a production SNI splitter every peer looks like <code>127.0.0.1</code> , so <code>allow 127.0.0.1</code> would expose <code>stub_status</code> to the internet.

9 · Security hardening

Everything below is emitted into the generated vhost. Defaults are chosen to be safe on an app nobody has audited: headers that cannot break a normal site are **on**; anything that can (CSP, origin lockdown, PROXY protocol) is **off** until switched on deliberately.

Control	Default	What it stops
httproxy guard EDGE_HTTPPROXY_GUARD	on	<code>fastcgi_param HTTP_PROXY ""</code> ; (nginx) / <code>RequestHeader unset Proxy early</code> (Apache). A <code>Proxy:</code> request header otherwise becomes <code>\$_SERVER['HTTP_PROXY']</code> and reroutes the app's outbound HTTP through an attacker's proxy (CVE-2016-5385). The distro <code>fastcgi_params</code> does not clear it.
429 on throttle	on	<code>limit_req_status 429</code> . nginx's default 503 was being mapped to <code>/50x.html</code> , so a rate-limited visitor saw the server-error page and the event looked like an outage.
Header stripping EDGE_HIDE_HEADERS	X-Powered-By,...	PHP's <code>expose_php</code> sends the exact patch level straight through the proxy. Removed via <code>fastcgi_hide_header</code> / <code>proxy_hide_header</code> / <code>Header always unset</code> .
Isolation headers EDGE_PERMISSIONS_POLICY , EDGE_COOP , EDGE_CORP	on	Device APIs the app never asked for; cross-origin references that enable Spectre-class leaks and unwanted embedding. Set any to empty to omit it.
CSP EDGE_CSP , EDGE_CSP_REPORT_ONLY	off	The strongest anti-XSS control, but app-specific — a wrong policy breaks the site, so nothing is invented. Start with report-only.
Slow-request guards EDGE_TIMEOUTS , EDGE_CLIENT_*_TIMEOUT	on	Slowloris / slow POST. nginx ships 60s header+body timeouts; a few dozen dribbling sockets can hold every worker. Apache gets <code>RequestReadTimeout</code> (gated on <code>mod_reqtimeout</code>).
Body ceiling EDGE_MAX_BODY	25m	Bounds what one request can make the box buffer. Was hardcoded; now per-deployment, and converted to bytes for Apache's <code>LimitRequestBody</code> .
Deny lists EDGE_DENY_EXT , EDGE_DENY_FILES	extended	<code>location /</code> ends in <code>try_files \$uri</code> , which serves <i>any</i> existing file — so this list is the only guard. Now covers private keys (<code>pem key crt p12...</code>), databases and archives (accidental backup dumps), plus build metadata by name (<code>composer.lock</code> , <code>Dockerfile</code> ...). Deliberately not a blanket <code>.json</code> : Vite manifests and <code>.well-known/*.json</code> are legitimate.
Upload guard EDGE_UPLOAD_MODE , EDGE_UPLOAD_PATHS	attachment	Stored XSS. <code>storage</code> is intentionally not denied (the <code>public/storage</code> symlink), so an uploaded <code>.svg</code> — which can carry <code><script></code> — or <code>.html</code> would run same-origin. Forced to download plus a <code>sandbox</code> CSP, or denied outright.
PROXY protocol EDGE_PROXY_PROTOCOL	off	See below — the client-IP collapse behind the SNI splitter.
Origin lockdown EDGE_CLOUDFLARE_ONLY , EDGE_ORIGIN_PULL_CA	off	Direct-to-origin traffic that bypasses the WAF. Either allow only Cloudflare's ranges, or require a client certificate from Cloudflare's origin-pull CA (mTLS).
OCSP resolver EDGE_RESOLVER	set	<code>ssl_stapling on</code> without a resolver silently never staples — nginx just logs and carries on.

9.1 · Client IPs behind the SNI splitter

A plain L4 `proxy_pass` rewrites the source address, so every visitor reached the backend as `127.0.0.1`. Two consequences: per-IP rate limiting became a single shared bucket, and — with the Cloudflare real-IP prelude on — `127.0.0.1` is a *trusted* proxy, so any client could send `CF-Connecting-IP: <anything>` and choose its own rate-limit identity.

`EDGE_PROXY_PROTOCOL=1` emits **both ends together** — `proxy_protocol on` in the stream server and `listen ... proxy_protocol + real_ip_header proxy_protocol` on the backend — because enabling one

side alone breaks every request. It is never emitted on a directly-reachable vhost, which would reject ordinary clients.

Behind Cloudflare the recovered address is the **CF edge**, not the visitor: nginx cannot chain two real-IP passes. That is still an address the client cannot choose. For true per-visitor limiting behind Cloudflare, rate-limit at the edge or don't put the L4 splitter in front of CF-fronted domains.

9.2 · Apache parity

The Apache vhost previously carried HSTS and nothing else. It now emits the same security headers, the same directory/extension/filename denials, dotfile denial for **files** as well as directories (`<DirectoryMatch "/\.">` alone misses `/.env`), a method allowlist, `AllowOverride None` , per-site logs, TLS protocol/cipher pinning, and — the biggest change — **only `index.php` executes**. A blanket `<FilesMatch "\.php$">` `SetHandler` ran any `.php` that reached the docroot.

Directive contexts matter. `<LimitExcept>` is directory-context only and `TraceEnable` is server-config only — Apache *refuses to start* if either is placed in a `<VirtualHost>` . The method guard is therefore emitted inside `<Directory>` and `TraceEnable Off` once at file level. Everything module-dependent (`Header` , `RequestReadTimeout` , `SSLProtocol`) is gated on `apachectl -M` , because an unknown directive is a hard `configtest` failure.

10 · Configuration reference

All config is env-driven through `config/edge.php` , read via the `edge_config()` helper. It is backed by the compiled config manifest, so a project's `projects/<name>/config/edge.php` is **deep-merged** over the plugin default — overriding one upstream keeps the rest. Outside a booted kernel (unit tests) it falls back to the shipped array.

Ports & TLS

Env	Default	Purpose
<code>EDGE_LISTEN_PORT</code>	443	Public TLS port
<code>EDGE_HTTP_PORT</code>	80	Public plain-HTTP port (<code>tls=none / both</code>)
<code>EDGE_TLS_MODE</code>	ssl	Default TLS mode: <code>ssl none both</code>
<code>EDGE_SSL_CERT</code> / <code>EDGE_SSL_KEY</code>	<code>/etc/ssl/certs/hkm-edge.pem</code> / <code>.../private/hkm-edge.key</code>	Certificate paths
<code>EDGE_SSL_PROTOCOLS</code> / <code>EDGE_SSL_CIPHERS</code> / <code>EDGE_SSL_STAPLING</code>	<code>TLSv1.2 TLSv1.3</code> / <code>modern</code> / <code>false</code>	Explicit TLS pinning; keep stapling off for Cloudflare Origin CA
<code>EDGE_HSTS</code> · <code>_MAX_AGE</code> · <code>_SUBDOMAINS</code> · <code>_PRELOAD</code> · <code>_DEV_MAX_AGE</code>	<code>true / 31536000 / true / false / 300</code>	HSTS, profile-aware; preload is opt-in

Strategy & upstreams

Env	Default	Purpose
EDGE_FORCE_STRATEGY	<i>empty</i>	Pin <code>nginx-only</code> <code>apache-only</code> ; empty = auto-detect
EDGE_REUSE_STREAM	<code>true</code>	Reuse an existing SNI splitter instead of writing a second one
EDGE_STREAM_BACKEND	<code>nginx_backend</code>	Upstream name the merged domains map to
EDGE_NGINX_BACKEND / EDGE_APACHE_BACKEND	<code>127.0.0.1:444</code> / <code>127.0.0.1:8443</code>	Stream upstreams (also sets the internal vhost port)
EDGE_APP_BACKEND	<code>127.0.0.1:8080</code>	App upstream for nginx-only / Apache
EDGE_NGINX_SSL_PORT / EDGE_BEHIND_SNI_ROUTER	<i>auto</i> / <code>false</code>	Port the nginx-only vhost listens on when something else owns <code>:443</code>

Paths, reload commands & domains

Env	Default	Purpose
EDGE_STREAM_PATH / EDGE_NGINX_PATH / EDGE_APACHE_PATH	<code>var/edge/*.conf</code>	Where each config is written — point at <code>/etc/nginx/...</code> in production
EDGE_RELOAD	<code>false</code>	Reload after write by default
EDGE_NGINX_TEST_CMD / _RELOAD_CMD	<code>nginx -t</code> / <code>nginx -s reload</code>	Validate/reload nginx
EDGE_APACHE_TEST_CMD / _RELOAD_CMD	<code>apachectl configtest</code> / <code>apachectl graceful</code>	Validate/reload Apache
EDGE_ALLOWED_BINARIES	<i>empty</i>	Extend the shell allow-list for a distro-specific binary
EDGE_EXTRA_DOMAINS / EDGE_EXCLUDE_DOMAINS	—	Comma-separated add / drop
EDGE_LOCAL_TLDS	<code>local,test,localhost,example,invalid</code>	TLDs treated as local
EDGE_MANAGE_HOSTS · EDGE_HOSTS_PATH · EDGE_HOSTS_IP	<code>true</code> · <code>/etc/hosts</code> · <code>127.0.0.1</code>	The hosts sync
EDGE_LOCAL_IN_SERVER	<code>false</code>	Serve local domains from the vhost outside dev too
EDGE_PROJECTS_DIR	<i>resolved</i>	Override where <code>projects.json</code> is read from

Serve model, env injection, caching & hardening

Env	Default	Purpose
EDGE_SERVE_MODEL	php-fpm	Default runtime (php-fpm openswoole)
EDGE_FPM_SOCKET	auto	Pin the FPM socket instead of auto-resolving
EDGE_SWOOLE_HOST / _PORT / _WORKERS / _BALANCE / _KEEPALIVE* / _MAX_FAILS / _FAIL_TIMEOUT / _PHP / _COMMAND / _WS_PATH	127.0.0.1 / 9501 /...	OpenSwoole upstream + unit defaults
EDGE_INJECT_KERNEL_ENV / EDGE_APP_ENV	true / APP_ENV → production	Run-env injected into every vhost
EDGE_COMPRESSION	auto	auto brotli gzip off
EDGE_CACHE_ASSETS / _TTL / EDGE_CACHE_HTML / EDGE_CACHE_CLOUDFLARE	true / 31536000 / false / true	Browser caching
EDGE_HTTP_PRELUDE · EDGE_LOG_* · EDGE_RATE_* · EDGE_CLOUDFLARE_*	false · ... · 10r/s / 50 · true	File-level log format, rate-limit zones, real-IP restore
EDGE_CORS · _ORIGINS · _METHODS · _HEADERS · _CREDENTIALS	off	CORS policy
EDGE_ALLOWED_METHODS / EDGE_DENY_DIRS	see §8	Method guard and denied directories
EDGE_PER_SITE_LOGS / EDGE_NGINX_DEBUG_LOG / EDGE_NGINX_STATUS / EDGE_DEV_VHOST	true / false / true /follows APP_ENV	Logging and dev extras

11 · Safety mechanisms

Mechanism	What it prevents
Binary allow-list in <code>SystemProbe::run()</code>	Reload commands come from env and typically run as root . Env is a weaker trust boundary than code, so the effective binary (skipping <code>sudo</code> , its flags and <code>F00=bar</code> prefixes, reduced to a basename) must be in <code>ALLOWED_BINARIES</code> — <code>nginx</code> , <code>apache2ctl</code> , <code>apachectl</code> , <code>httpd</code> , <code>systemctl</code> , <code>service</code> , <code>rc-service</code> , <code>brew</code> , <code>command</code> , <code>test</code> , <code>true</code> . Commands containing <code>;</code> <code>&</code> <code> </code> <code>`</code> <code>\$</code> <code>(</code> are rejected outright, since only the first binary is validated. Refusal returns exit 126 with an explanation.
Hostname validation	Every domain is checked against a strict charset regex before it can reach a rendered config — a malformed <code>proj.json</code> entry can never inject directives.
Atomic writes everywhere	Temp file + rename for the config, the hosts block and the stream map, so a live <code>include</code> never sees a half-written file.
Backup + rollback	A failed config test restores the previous file instead of leaving a broken config that the next reload would load.
Marker-delimited edits	Both the hosts block and the stream sub-block are fully rewritten between markers; everything outside is preserved verbatim.
Existing entries win	A host mapped by hand — in <code>/etc/hosts</code> or in your SNI map — is skipped, never duplicated or repointed.
Dev gate on <code>/etc/hosts</code>	A VPS run never touches the machine's hosts file.
Fail-safe cache profile	Only an explicit production <code>APP_ENV</code> yields the <code>PRODUCTION</code> profile — a stale asset in dev is cheap, a year of wrong immutable caching is not.
Fail-loud stream merge	A map write that could not complete (missing <code>sudo</code>) fails the entire apply rather than reporting success.
Capability-aware output	Brotli degrades to gzip when the module is absent; Apache features check <code>apachectl -M</code> (unknown ⇒ assume present, with <code>configtest</code> as the backstop) — so the generated config never fails the server's own test.

12 · Recipes & troubleshooting

```
# Local dev – serve *.local sites via nginx/Apache + /etc/hosts
sudo hkm cli -p shop --dev edge:apply --local

# Preview only, write nothing
hkm cli -p shop edge:status
hkm cli -p shop edge:apply --production --dry-run

# Production VPS with TLS (paths via EDGE_*_PATH)
hkm cli -p shop edge:apply --production

# Production, NO certificate on the host – plain HTTP, nginx as sole front
hkm cli -p shop edge:apply --production --no-ssl --nginx-only

# Redirect all HTTP to HTTPS (ACME challenge still passes through)
hkm cli -p shop edge:apply --production --tls=both

# Bring an OpenSwoole project under systemd
sudo hkm cli -p shop edge:service --production --write
sudo systemctl daemon-reload && sudo systemctl enable --now hkm-shop

# Every registered project at once
hkm cli -p shop edge:apply --production --all

# Hosts file
sudo hkm cli -p shop --dev edge:hosts
sudo hkm cli -p shop --dev edge:hosts --remove
```

Symptom	Cause / fix
nginx -t fails on the certificate	Default <code>ssl</code> mode expects a cert. Use <code>--no-ssl</code> (or <code>EDGE_TLS_MODE=none</code>) on a host with none. The SNI splitter cannot apply without TLS, so pair it with <code>--nginx-only</code> .
nginx fails with “Address already in use”	Something else (an SNI router) owns <code>:443</code> . Auto-detected, but you can force with <code>EDGE_BEHIND_SNI_ROUTER=1</code> or pin <code>EDGE_NGINX_SSL_PORT</code> .
“stream map update failed ... need sudo?”	Editing <code>nginx.conf</code> needs root. Re-run with <code>sudo</code> , or set <code>EDGE_REUSE_STREAM=0</code> to have Edge write its own block.
“refusing to touch the hosts file outside dev mode”	Run via <code>hkm ... --dev</code> , or pass <code>--force</code> if you really mean it.
Duplicate <code>log_format</code> / <code>limit_req_zone</code> error	Your <code>nginx.conf</code> already declares them — keep <code>EDGE_HTTP_PRELUDE=false</code> (the default).
Site 404s on static files	Static assets resolve only under <code><project>/app/public</code> ; a miss is a deliberate hard 404 and is never forwarded to the app.
The vhost boots the wrong kernel	FPM workers don't inherit the launcher env. Check the injected <code>fastcgi_param</code> set and whether <code>EDGE_INJECT_KERNEL_ENV</code> is on and the keys were actually exported.
Nothing generated for a project	A <code>Site</code> is skipped when it has no valid domain or no path. Check <code>proj.json domains[]</code> and the validation charset.

13 · Observations from the code read

1 · Declared-config drift — fixed. `module.json` declared 63 env keys while `config/edge.php` read 21 more (`EDGE_FORCE_STRATEGY` , `EDGE_REUSE_STREAM` , all `EDGE_CORS_*` , ...). The manifest now declares all **114** keys the module actually reads, so it is a complete inventory of the plugin's knobs.

2 · Documentation drift on defaults. `README.md` lists `EDGE_SWOOLE_BASE_PORT` with default 9500 and `EDGE_SERVE_MODEL` default `fpm` ; `USAGE.md` and the code use `EDGE_SWOOLE_PORT` (9501) and `php-fpm` . `USAGE.md` is the accurate one.

3 · Swoole command default differs by source. `SwooleOptions` 's constructor default is `app/swoole/index.php` , while `SiteCollector` falls back to `edge_config('serve.swoole_command'` , `'bin/server.php')` . In practice the config path always wins, so the constructor default is effectively dead — but the two disagreeing defaults are a trap for a future reader.

Generated from a full read of the plugin source — `Provider.php` , `module.json` , `config/edge.php` , the contract, all 9 Domain classes, `EdgeService` , the 6 Infrastructure classes and the 4 CLI commands — plus `README.md` and `USAGE.md` .